CSCE 775: Deep Reinforcement Learning Comprehensive Study Plan

Lectures 4–7: Model-Free RL, Deep Learning, PyTorch, DAVI & DQNs

J.C. Vaught · Spring 2026 · University of South Carolina

Contents

Area 1: Model-Free Prediction (MC & TD Methods)

Textbook Exercises (Sutton & Barto, 2nd Ed.)

Chapter 5: Monte Carlo Methods

Priority exercises: 5.1–5.6, and especially **5.12** (programming).

Ex.	Description
5.1	Analyze why value function plots for blackjack show jumps at certain states
5.2	Compare bias properties of every-visit and first-visit MC estimators
5.3	Draw and describe the backup diagram for Monte Carlo estimation of $Q(s, a)$
5.4	Modify MC ES pseudocode to use incremental mean updates instead of storing all returns
5.5	Calculate both first-visit and every-visit estimator types on a concrete example
5.6	Derive importance sampling equations for action value estimates $Q(s, a)$
5.7	Explain the behavior of MSE in weighted importance sampling at the start of learning
5.8	Determine variance properties of the every-visit MC estimator
5.9	Modify the off-policy MC algorithm using incremental update formulas
5.10	Derive the recursive weighted-average update rule
5.11	Explain why the importance sampling ratio starts from $t+1$ in off-policy control
5.12	**Racetrack problem**: Implement MC control for a racetrack gridworld with noisy acceleration

Chapter 6: Temporal-Difference Learning

Priority exercises for prediction: 6.1, 6.2, 6.3, 6.4, 6.5, 6.6.

Ex.	Description
6.1	Derive the telescoping sum identity for TD errors (fundamental)
6.2	The "buffet" thought experiment—explain when TD outperforms MC
6.3	Compute value updates after the first episode of the 5-state random walk
6.4	Analyze how different α values affect TD vs. MC performance
6.5	Investigate how increasing learning rate affects MSE over multiple episodes
6.6	Describe methods for computing the exact values in the random walk MRP

Chapter 7: n-step Bootstrapping

Both exercises are essential. Exercise 7.2 gives hands-on experience with the bias-variance spectrum.

Ex.	Description
7.1	Show the n-step return error can be written as a sum of TD errors
7.2	**Programming**: Compare n-step TD methods for different n on the 19-state random walk, reproducing Figure 7.2

University Assignments

Assignment	What You Do
David Silver's Easy21 (UCL)	The gold standard. 5 parts: implement Easy21 env, MC control, SARSA(λ), linear FA, discussion. Assignment PDF
Georgia Tech CS 7642 Project 1	Reproduce Sutton (1988) original TD experiments on bounded random walk. Course page
Stanford CS234 Module 3	TD policy evaluation, batch MC vs batch TD. Course page

Mini-Projects

Project A: MC vs TD(0) Random Walk Comparison

Implement a configurable N-state random walk MRP (support $N = 5, 19, 99$). Implement first-visit MC prediction and TD(0) prediction. Reproduce Figure 6.2 from Sutton and Barto: plot RMS error vs. episodes for TD(0) with $\alpha \in \{0.05, 0.1, 0.15\}$ and constant-α MC with $\alpha \in \{0.01, 0.02, 0.04\}$. Reproduce Example 6.3 (batch updating). Extend to 19 states and reproduce Figure 7.2.

Deliverables: Four publication-quality figures, a written analysis (1–2 pages) of the bias-variance tradeoff observed.

Project B: Blackjack Policy Evaluation Showdown

Use Gym's Blackjack-v0 environment. Implement first-visit MC, every-visit MC, TD(0), and n-step TD prediction for the fixed policy: stick on 20 or 21, hit otherwise. Measure convergence speed. Produce 3D surface plots of the value function at 10K, 100K, and 500K episodes.

Key Coding Resources

- Denny Britz RL Repository – MC/TD notebooks with starter code and solutions
- Shangtong Zhang Figure Reproductions – Python reproductions of nearly every textbook figure
- Udacity Deep RL MC Notebook

Recommended Sequence

1. Read S&B Ch 5.1–5.5, do exercises 5.1–5.6
2. Code Denny Britz MC Prediction notebook (Blackjack)
3. Read Ch 6.1–6.3, do exercises 6.1–6.6
4. Code Mini-Project A (random walk, Figures 6.2 and 7.2)
5. Read Ch 7.1–7.2, do exercises 7.1–7.2
6. Complete Easy21 Parts 1–3

Area 2: Model-Free Control (SARSA, Q-Learning, On/Off-Policy)

Textbook Exercises (Chapter 6)

Priority: 6.6–6.7 (Windy Gridworld + extensions), 6.9 (Expected SARSA on cliff walking), 6.11–6.12 (why Q-learning is off-policy; greedy SARSA vs Q-learning), 6.14 (maximization bias / Double Q-learning).

Ex.	Description
6.6	Implement SARSA on Windy Gridworld (Example 6.5); plot episodes vs time steps
6.7	Extend 6.6 with King's moves (8 actions) and stochastic wind
6.8	Derive the relationship between returns and action-value estimates through TD decomposition
6.9	Programming: Compare SARSA, Q-learning, and Expected SARSA on cliff walking
6.10	Why Q-learning's policy (argmax) differs from its behavior policy (ε-greedy)
6.11	Explain precisely why Q-learning is classified as off-policy control
6.12	"If action selection is greedy, is Q-learning exactly the same as SARSA?"
6.13	Explore double learning applied to Expected SARSA
6.14	Programming: Maximization bias problem, Q-learning vs Double Q-learning

University Assignments

Assignment	What You Do
Easy21 Parts 3–5	SARSA(λ) with eligibility traces, linear FA comparison, discussion
Stanford CS234 Asgn. 2	Linear Q-learning then full DQN on Atari. Course page
Berkeley CS285 HW3	DQN + Double DQN on LunarLander and Ms. Pac-Man. Course page

Classic Environments to Implement

Cliff Walking (Example 6.6)

4×12 grid. Start at bottom-left, goal at bottom-right. The bottom row between start and goal is a "cliff"—stepping on it gives reward -100 and resets to start. Every other step gives -1. This is THE canonical demonstration of on-policy vs off-policy behavior. SARSA accounts for its own exploration noise; Q-learning ignores it.

Implement SARSA, Q-learning, and Expected SARSA. Plot sum-of-rewards per episode for all three algorithms over 500 episodes, averaged over 100 runs.

Windy Gridworld (Example 6.5)

7×10 grid with wind columns. Implement SARSA with 4 actions, then extend to King's moves (8 actions), stochastic wind. Plot timesteps vs episodes completed.

Taxi-v3 (Gymnasium)

500 discrete states. 6 actions. Compare Q-learning, SARSA, and Expected SARSA convergence rates and final win rates. Environment docs.

Maximization Bias / Double Q-Learning (Example 6.7)

Implement Q-learning (observe overestimation) and Double Q-learning (observe correction). Plot % of left actions taken from start state over episodes.

Mini-Projects

Project 1: TD Control Benchmark

Implement SARSA, Expected SARSA, Q-learning, and Double Q-learning from scratch (no RL libraries). Benchmark on FrozenLake-v1 (4×4 and 8×8, slippery=True), CliffWalking-v0, and Taxi-v3. For each algorithm × environment: run 100 independent trials, 1000 episodes each. Vary $\varepsilon \in \{0.01, 0.05, 0.1, 0.2\}$ and $\alpha \in \{0.1, 0.3, 0.5\}$. Produce heatmaps of final average return.

Project 2: Safety-Aware Navigation

Design a custom 10×10 gridworld with danger zones (−50 but non-terminal) and cliffs (−100, reset). Demonstrate SARSA avoids danger zones during training while Q-learning finds the shorter but riskier path. Visualize learned Q-value heatmaps and policy arrows.

Key Coding Resources

- Denny Britz SARSA/Q-learning notebooks
- Cliff Walking implementations
- Windy Gridworld implementations
- RL Sketchpad – Ch. 6 figure reproductions

Recommended Sequence

1. Read S&B Ch 6.4–6.5 (SARSA, Q-learning). Do exercises 6.8, 6.11, 6.12
2. Implement Windy Gridworld (6.6–6.7)
3. Implement Cliff Walking—reproduce Example 6.6
4. Implement Double Q-learning (6.14) and Taxi-v3
5. Complete Easy21 all parts
6. Bridge to deep RL with CS285 HW3 or CS234 Assignment 2

Area 3: Deep Learning Foundations

University Assignments

Assignment	What You Do
CS231n Assignment 1	k-NN, SVM, Softmax, Two-layer net—all from scratch in NumPy on CIFAR-10. Assignment page
CS230 Section 3 Exercises	Hand-worked backprop through computation graphs. Exercises PDF
Andrew Ng's DL Specialization	Course 1 Weeks 2–4: logistic regression, 1-hidden-layer net, L-layer net from scratch. Course 2 Week 1: L2 reg, dropout, gradient checking. Course page
CS229 Problem Set 1	Logistic regression, gradient/Hessian derivation, Newton's method vs GD. PDF

Hand-Computation Exercises (Do on Paper)

1. **Matt Mazur's Step-by-Step Example**: 2-input, 2-hidden, 2-output net with concrete numbers. Full forward+backward pass. Link
2. **Single-neuron logistic regression**: Derive $\partial L/\partial w$ showing the form $(\sigma(z) - y) \cdot x$
3. **Two-layer network with MSE**: Derive all four gradient terms $\partial L/\partial W_2$, $\partial L/\partial b_2$, $\partial L/\partial W_1$, $\partial L/\partial b_1$
4. **ReLU vs. Sigmoid gradient flow**: Repeat exercise 3 with ReLU. Notice why ReLU mitigates vanishing gradients
5. **Softmax + Cross-Entropy**: Show gradient simplifies to $(softmax(z) - y_{one-hot})$

From-Scratch Projects

Project 1: Linear Regression from Scratch

Pure NumPy. Implement forward pass ($\hat{y} = Xw+b$), MSE loss, gradient computation, and gradient descent update loop. Compare with the closed-form solution (normal equation). Extend to batch, stochastic, and mini-batch gradient descent.

Project 2: Logistic Regression from Scratch

Binary classifier on `make_moons`. Implement sigmoid, binary cross-entropy, backward pass, training loop. Visualize decision boundary at different epochs. Add L2 regularization and observe changes.

Project 3: Karpathy's Micrograd (Rebuild It)

The single best exercise for understanding backpropagation and autograd. Build a scalar-valued automatic differentiation engine (∼100 lines of Python). Implement a `Value` class, `backward()`

using topological sort, then `Neuron`, `Layer`, and `MLP` classes.

- Repository: github.com/karpathy/micrograd
- Video lecture (2+ hours): karpathy.ai/zero-to-hero.html

Project 4: Two-Layer Neural Network from Scratch

Full NumPy implementation on MNIST. Implement parameter initialization, forward pass, cross-entropy loss, backward pass, and mini-batch training loop. Extensions: ReLU activation, L2 regularization, dropout, momentum, and Adam optimizer.

Project 5: Michael Nielsen's 74-Line Network

Work through Chapters 1–3 of neuralnetworksanddeeplearning.com. Classifies MNIST digits at >96% accuracy with no frameworks.

Intuition-Building Mini-Projects

- **Loss Landscape Visualization**: Visualize the loss surface along two random directions in parameter space. Reference: Li et al., 2017
- **Gradient Flow Monitor**: Track gradient magnitudes per layer. Compare sigmoid (vanishing) vs. ReLU (healthy) vs. tanh. Add batch normalization.
- **Optimizer Comparison**: Implement vanilla SGD, momentum, RMSProp, Adam on the Rosenbrock function. Plot trajectories on contour plots.
- **Overfitting Laboratory**: Fit progressively larger networks on 20 points from a noisy sine wave. Then add L2, dropout, early stopping one at a time.

Recommended Sequence

1. Hand-compute backprop (Matt Mazur, CS230 exercises)
2. Implement logistic regression from scratch
3. Build micrograd—the most important single exercise
4. CS231n Assignment 1 (two-layer net portion)
5. Nielsen Chapters 1–3

Area 4: PyTorch & Automatic Differentiation

Official PyTorch Tutorials (In Order)

1. Learning PyTorch with Examples
2. Gentle Introduction to torch.autograd
3. Fundamentals of Autograd (video)
4. Optimizing Model Parameters
5. Autograd Mechanics Reference

Build Autograd from Scratch

Resource	Description
Karpathy's micrograd	Scalar autograd engine + MLP (∼100 lines). GitHub
CMU 11-785 new_grad	Tensor-valued autodiff on NumPy—closer to real PyTorch. GitHub
CMU 11-785 MyTorch HW series	Build PyTorch from scratch over 4 assignments (linear layers, batch norm, conv, RNNs). Course page

University Assignments

Assignment	What You Do
UMich EECS 498 Asgn. 4	PyTorch Autograd and nn—three abstraction levels (raw tensors, `nn.Module`, `nn.Sequential`), culminates in ResNet. Assignment page
CS231n Assignment 2	Batch normalization, dropout, conv layers from scratch, then training with PyTorch. Assignment page
D2L.ai Ch 2.5	Autograd exercises with PyTorch. Link

Mini-Projects

1. **MNIST CNN**: Build with raw `nn.Conv2d`, `nn.BatchNorm2d`, `nn.Linear`. Write full training loop manually. Target >99% accuracy.
2. **ResNet-18 from Scratch on CIFAR-10**: Implement `BasicBlock` with skip connections. Compare with/without residual connections.
3. **Normalization Comparison**: Replace BatchNorm with LayerNorm, GroupNorm, InstanceNorm in your ResNet. Compare convergence.

Supplementary Resources

- Understanding PyTorch Computational Graphs (blog with diagrams)

- UW CSE 599W Lecture 4: Backprop and Autodiff (systems perspective)
- U Toronto CSC421: Automatic Differentiation (theoretical)
- fast.ai Part 2: 30+ hours building Stable Diffusion from scratch in PyTorch

Recommended Sequence

1. Official PyTorch tutorials 1–5
2. Build micrograd from scratch
3. CMU 11-785 HW1 Part 1 (MyTorch linear layers)
4. Mini-Projects 1 and 2 (MNIST CNN, ResNet-18)
5. UMich EECS 498 Assignment 4

Area 5: DAVI and DQNs

Papers to Read (In Order)

1. **Mnih et al. (2013)**: Original DQN. arXiv:1312.5602
2. **Mnih et al. (2015)**: Nature DQN (adds target networks). Nature
3. **Agostinelli et al. (2019)**: DeepCubeA—uses DAVI + batch weighted A* search. Nature MI. Code: GitHub
4. **van Hasselt et al. (2016)**: Double DQN. arXiv:1509.06461
5. **Schaul et al. (2016)**: Prioritized Experience Replay. arXiv:1511.05952
6. **van Hasselt et al. (2018)**: Deadly Triad. arXiv:1812.02648

University Assignments

Assignment	What You Do
Stanford CS234 Asgn. 2	Linear Q-learning to full DQN on Atari (experience replay + target networks). Course page
Berkeley CS285 HW3	DQN + Double DQN on LunarLander and Ms. Pac-Man. Course page
CMU 10-703 HW2	Deep Q-networks with online + target networks. Course page

Progressive Implementation Path

Level	Project	Environment
1	Tabular Q-learning	FrozenLake-v1
2	Linear Q-learning (observe instability)	CartPole-v1
3	Full DQN (replay, target net, ε-greedy)	CartPole-v1
4	DQN with tuning	LunarLander-v2
5	DQN on pixels (CNN, frame stacking)	Pong / Breakout
6	Double DQN + Prioritized Replay	Atari
7	**DAVI on combinatorial puzzle**	15-puzzle or 2×2 Rubik's cube

Key Concepts

Target Networks

If the same network generates both the prediction and the target, every update shifts the target, creating a feedback loop. The target network θ^- freezes the target for N steps. The DQN loss becomes: $\mathrm{L}(\theta) = E\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right]$

Experience Replay

Online RL training data is (a) temporally correlated and (b) non-stationary. Both properties violate the i.i.d. assumption of SGD. Experience replay stores transitions (s, a, r, s') in a buffer and samples mini-batches uniformly at random, breaking both correlations.

The Deadly Triad

Function approximation + bootstrapping + off-policy learning can cause divergence. DQN mitigates this with target networks and experience replay, but does not fully solve it.

Deep Approximate Value Iteration (DAVI)

Generate training data by scrambling from the goal state. Train a neural network to predict cost-to-go. Use that network as a heuristic in A* search. This is the approach in Prof. Agostinelli's DeepCubeA paper.

Key Implementation References

- CleanRL DQN—single-file, most readable DQN
- PyTorch DQN Tutorial—CartPole from scratch
- Hugging Face Deep RL Course Unit 3—theory + hands-on notebook
- Antonin Raffin: Tabular to DQN—best progressive walkthrough
- Daniel Takeshi: Atari Preprocessing

Recommended Sequence

1. Read Sutton & Barto Ch. 9–11 (function approximation theory)
2. Read DQN papers (2013, then 2015 Nature)
3. Implement tabular Q-learning on FrozenLake, then linear FA on CartPole
4. Implement full DQN on CartPole, then LunarLander
5. Read DeepCubeA paper, study the GitHub repo
6. Implement simplified DAVI on a combinatorial puzzle

Solution References for Self-Checking

Resource	Link
Sutton & Barto Solutions (LyWangPX)	GitHub
Tianlin Liu's Solutions PDF	PDF
Scott Jeen's Exercise Compilation	PDF
Easy21 Solutions (alexandrasouly)	GitHub
CS234 Solutions (ksang)	GitHub
CS285 Solutions (ZHZisZZ)	GitHub
Denny Britz RL Repository	GitHub
Shangtong Zhang Reproductions	GitHub

---

*The single assignment that ties Areas 1 and 2 together most effectively is **Easy21**. For Areas 3 and 4, **micrograd** is the highest-value single exercise. For Area 5, start with CleanRL's DQN and work up to studying Prof. Agostinelli's DeepCubeA code.*